Versión 1.3 — 3 de octubre de 2026

Unidades: 0.1, 0.2, 0.3, 1.0 (subitizing), 2.1 (contar), 2.2 (la lista), 2.3 (cuántos hay) y 2.4 (cardinalidad)

> **Arreglo de esta versión (fallo que encontraste tú):** `contar:` contaba **cinco** objetos en

> `casa(color=blanco), casa, perro, casa(forma=cuadrado), carro`, cuando son **tres**: la casa blanca y

> la casa cuadrada son **la misma casa** (sus datos se suman) y `perro` y `carro` son otras dos. No

> aplicaba la regla de identidad que fijaste en 0.1, y ni siquiera leía los atributos: tomaba

> `casa(color=blanco)` como si fuera el nombre entero. Ya está corregido, con tu caso metido en las

> pruebas. Aquí no se borra nada: se arregla y se deja escrito.

Novedad de la versión 1.3: la unidad 2.4 — cardinalidad

El sistema puede producir un conjunto con la cantidad que le pidas. Y eso es el **principio**, no la regla de la 2.3.

```
dame: tres

  CONJUNTO QUE DOY (3 objetos):
    cosa_uno
    cosa_dos
    cosa_tres

  VERIFICACION (lo cuento otra vez): la ultima palabra es 'tres'
  cuadra con lo pedido: SI
```

Por qué dar es más difícil que reconocer

Reconocer una cantidad (conjunto → palabra, la 2.3) es más fácil que **producirla** (palabra → conjunto). En los estudios hay niños que cuentan bien hasta cinco y **no saben dar cinco objetos**. Por eso esta tarea —**Give-N**— es la que distingue de verdad, y es la que usa esta unidad.

Cuatro cosas que hace, y todas salen de la investigación

- **Se verifica a sí mismo.** Cuenta lo que ha dado y comprueba que cuadra. (En los estudios, pedir que se rechequee mejora los resultados.)
- **Es un *counter*, no un *grabber*.** No agarra un puñado y dice una cantidad: cuenta.
- **No se lo inventa.** `dame: once` → `palabra_desconocida`.
- **Y "¿tiene N?" confirma o rechaza**: `tiene: a,b,c / tres` → **sí**; `tiene: a,b,c / cinco` → **no**, y dice que tiene tres.

La prueba que más vale (coherencia para TODA la lista)

Para cada palabra: `dame: N` → contar lo dado → tiene que dar **N**. De «uno» a «diez», cuadra.

```
OK  dame:uno   ->  1 objeto  -> cuantos: uno
OK  dame:dos   ->  2 objetos -> cuantos: dos
...
OK  dame:diez  -> 10 objetos -> cuantos: diez
```

Las dos direcciones están de acuerdo para **toda** la lista. Eso es el principio funcionando.

Novedad de la versión 1.2: la unidad 2.3 — cuántos hay

El sistema ya dice cuántos hay.

```
cuantos: silla_1, silla_2, silla_3

COLECCION:
    silla_1
    silla_2
    silla_3
  RECUENTO (una palabra por objeto, en orden):
    silla_1    -> uno
    silla_2    -> dos
    silla_3    -> tres

  la ULTIMA palabra del recuento es: tres
RESULTADO: hay_cantidad
```

La respuesta es **la última palabra** — y el sistema lo dice así, no dice «sé que hay tres».

Por qué esa frase tan retorcida (y no es un capricho)

La investigación separa dos cosas que parecen la misma:

- **La regla de la última palabra**: «la respuesta a ¿cuántos hay? es la última palabra». Es un **procedimiento**. Se puede aplicar bien sin entender nada.
- **El principio de cardinalidad**: «esa palabra **es** la cantidad del conjunto». Es el **principio**.

El dato que obliga a separarlas: en un estudio con **188 niños de 4 años y medio**, 42 (un 22,3%) acertaban **diciendo la última palabra** y fallaban la correspondencia uno a uno al contar. O sea, se puede acertar aquí sin entender lo de abajo.

Así que la 2.3 es **la regla**, y la **2.4 será el principio**. No se cuela.

Y las cosas que NO hace

- **No cuenta dos veces.** Los niños, al oír «¿cuántos hay?», muchas veces **vuelven a contar**. El sistema no: ya contó, y da la respuesta.
- **No se inventa la cantidad** si hay más objetos que palabras conoce: `no_alcanza_la_secuencia`.
- **No cuenta dos veces el mismo objeto**: `cuantos: a,a,a,a,a` → **uno**.
- Y tu caso: `cuantos: casa(color=blanco), casa, perro, casa(forma=cuadrado), carro` → **tres**.

Novedad de la versión 1.1: la unidad 2.2 — la lista

El sistema ya sabe **la lista**, y la sabe de verdad: no de corrido desde el principio, sino contestando **qué va antes y después de una palabra cualquiera**.

```
siguiente: siete      ->  ocho      (por el siete, no por el uno)
anterior: nueve       ->  ocho      (también hacia atrás)
recitar: ocho         ->  ocho, nueve, diez
                          Llego hasta el final de lo que se: no hay más después de 'diez'.
```

Y aquí está la gracia: al llegar al final, no se inventa nada

```
siguiente: diez       ->  no_lo_se
                          'diez' es la ULTIMA palabra que conozco. No me invento la siguiente.
siguiente: once       ->  palabra_desconocida
```

Los niños hacen exactamente esto: en los estudios, entre un 12% y un 25% dicen **«no lo sé»** cuando no saben. Ese «no lo sé» **no es un fallo**: es la respuesta correcta, y es la que este sistema da.

Y sabe revisar una recitación ajena

```
revisar_secuencia: uno,dos,cuatro   ->  orden_saltado   ("falta por decir: tres")
revisar_secuencia: uno,tres,dos     ->  orden_saltado   (los cambió de orden)
revisar_secuencia: uno,dos,once     ->  palabra_desconocida  (se inventó una al final)
revisar_secuencia: uno,dos,tres     ->  correcta
```

Una separación que quiero que sepas (porque es fácil de confundir)

Aquí NO se aprende que «cuatro es uno más que tres». Se aprende que después de «tres» va «cuatro» **porque está en la lista**. La investigación es clara: entender que cada palabra es «una más» (el sucesor) llega **después** de entender la cardinalidad. Así que aquí no se cuela, a propósito.

Novedad de la versión 1.0: la unidad 2.1 — CONTAR

Es la **primera unidad en la que el sistema deduce algo que no se le ha dado**. Hasta ahora, todo lo que contestaba estaba ya escrito en lo que le escribías. Ahora no: le das una colección y él produce el emparejamiento de una palabra con cada objeto.

```
contar: silla_1, silla_2, silla_3

COLECCION:
    silla_1
    silla_2
    silla_3
  EMPAREJAMIENTO (una palabra por objeto, ninguna repetida):
    silla_1    -> uno
    silla_2    -> dos
    silla_3    -> tres
RESULTADO: emparejado
```

Lo segundo, y lo más importante: sabe revisar una cuenta mal hecha

Se le da una colección y una cuenta que ha hecho otro, y el sistema dice si está bien y, si no, cuál es el error. Hay tres errores, y son los tres que la investigación documenta:

```
revisar: a,b,c / uno,uno,tres
  -> se uso la misma palabra mas de una vez: uno
RESULTADO: palabra_repetida          (el error MAS COMUN en los ninos)

revisar: a,b,c / uno,tres,cuatro
  -> falta por decir: dos
RESULTADO: orden_saltado

revisar: a,b,c / uno,dos
  -> quedaron objetos SIN contar: c
RESULTADO: faltan_objetos_sin_contar
```

Esto no se puede hacer por rutina: hay que tener el principio dentro.

Y una honestidad más: no se inventa las palabras que no conoce

El sistema sabe contar hasta **diez**. Si le das más objetos:

```
contar: a,b,c,d,e,f,g,h,i,j,k
RESULTADO: no_alcanza_la_secuencia
  Hay mas objetos que palabras conozco, y NO me invento las que faltan.
  (Aprender mas palabras de la secuencia es la unidad 2.2.)
```

Contar **no es recitar**: es emparejar. Y el sistema solo empareja con lo que tiene.

Lo que todavía NO hace (importante)

Contar no es todavía saber cuántos hay. El sistema **no dice «hay tres»**: pone las palabras, pero todavía no sabe que **la última palabra es el total**. Eso es la **cardinalidad** (unidad 2.4) y llega después — así es como se aprende: primero emparejar, luego entender que la última palabra resume todo.

Novedad de la versión 0.8: la unidad 1.0 — reconocer de un vistazo

Ya se puede enseñar una figura y preguntar cuántos elementos tiene, **de un vistazo**:

```
subitizar: x.x/.../x.x

FIGURA:
    x . x
    . . .
    x . x
  es el patron 'esquinas_cuatro'  ->  CUATRO
RESULTADO: reconocido
```

La figura se escribe con **tres filas separadas por** `/`, donde `x` es un elemento y `.` un hueco.

Lo importante: el sistema no cuenta

Y aquí está la gracia de la unidad. Si la figura **no** es un patrón conocido:

```
subitizar: x../xx./x.x
```

```
FIGURA:
    x . .
    x x .
    x . x
RESULTADO: no_reconocido
  No lo reconozco de un vistazo, y NO lo cuento: contar es la unidad 2.3.
  El sistema no adivina: si no lo reconoce, lo dice.
```

Esa figura tiene **cuatro** elementos. El sistema podría contarlos… si supiera contar, y todavía no sabe **(eso es el Nivel 2). Así que no lo finge: dice que no lo reconoce.** Reconocer no es contar.

Y la disposición es lo que hace reconocible a un grupo

- `x.x/.../x.x` → cuatro (esquinas, como un dado) ✔
- `xx./xx./...` → cuatro también: la misma cantidad en otra disposición vale lo mismo ✔
- `x../xx./x.x` → no se reconoce ✘

Esto no es un capricho: la investigación muestra que las disposiciones tipo dado se ven **enteras** y las desordenadas no, y que una fila recta de puntos es difícil de subitizar.

Las cantidades van en **palabras** (`uno`, `cuatro`), no en cifras: las cifras son la unidad 3.1.

Novedad de esta versión: el mismo nombre, dos objetos

Corregido a raíz de una prueba tuya. Antes, esto:

```
carro(color=negro), carro(color=blanco)
```

decía «es **un** objeto» y «se contradicen». Estaba mal, como tú dijiste: una misma cosa no puede ser negra y blanca a la vez**, así que un atributo con** dos valores distintos **es la** prueba de que son **dos objetos**. Ahora:

```
  AVISO: un mismo nombre con datos que NO pueden ser la misma cosa:
    carro  [color=negro]
    carro  [color=blanco]
  -> eso PRUEBA que son objetos DISTINTOS. Quedan asi: carro#1, carro#2
PLURALIDAD (0.1): son VARIOS OBJETOS
ATRIBUTOS (0.2): sin_atributo_comun
```

Las tres reglas del nombre, ya juntas:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa** (los datos se suman): `casa` blanca y cuadrada
`carro(color=negro); carro(color=blanco)`	**Dos objetos**: `carro#1` y `carro#2` — el color los distingue
`carro, carro`	**Un solo objeto**: sin atributos, nada los distingue

Novedad de esta versión: la unidad 0.3 — pertenencia

Ya se puede preguntar **si un objeto concreto pertenece a un grupo**:

```
pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)

  taza       -> PERTENECE
  plato      -> NO_PERTENECE
  vaso       -> NO_DECIDIBLE  (no se sabe su color)
```

Tres respuestas, no dos, y esa es toda la gracia:

- **PERTENECE** — el objeto tiene el color que pide el grupo.
- **NO_PERTENECE** — tiene color, pero **otro**.
- **NO_DECIDIBLE** — **no se sabe**: a ese objeto no le hemos dicho el color. Que no es lo mismo que «no pertenece»: para negar algo hay que saber que es falso, no basta con no saberlo.

Y algo que conviene probar: **un mismo objeto pertenece a varios grupos**. El triángulo verde pertenece a «los verdes» y a «los triángulos» — la pertenencia depende del criterio con el que se pregunta.

El Nivel 0 está completo

Con 0.1, 0.2 y 0.3 cerradas y verificadas, el sistema ya sabe: distinguir objetos, formar grupos por atributos y decidir pertenencia. **Todo rastreable y todo con casos nuevos probados.**

Novedad de esta versión: cada respuesta dice las DOS cosas

Cuando escribes objetos con atributos, el sistema contesta **en dos capas**:

- **PLURALIDAD (0.1):** cuántas cosas hay — `es UN OBJETO` o `son VARIOS OBJETOS`.
- **ATRIBUTOS (0.2):** si comparten algún atributo, y cuáles.

Así ya no esconde lo que sabe: `casa(color=azul); carro` **no** dice un «indefinido» a secas, sino «son **varios objetos**, y en cuanto a atributos **no se puede decidir** (del color del carro no sabemos nada)».

Y hay una regla nueva, la de la **fusión**:

Escribes	Qué entiende
`casa(color=blanco); casa(forma=cuadrada)`	**La misma casa**, descrita dos veces: sus datos se suman → `casa blanca y cuadrada`, **un solo objeto**
`casa(color=blanco); casa(color=rojo)`	**Se contradicen** (mismo atributo, dos valores) → no elige: `no_decidible`
`casa, casa`	**Un solo objeto** (el nombre manda) — si no, sería un grupo y eso rompería 0.1

Novedad de esta versión: el vocabulario, arreglado

Carlos destapó una contradicción: `casa,carro` decía *grupo*, y `casa(color=blanco); carro(color=negro)` decía *no grupo*. La misma pareja de objetos, respuestas opuestas.

La causa era usar la palabra **grupo** con dos sentidos distintos. Ya está separado:

Unidad	La pregunta que responde	Respuestas
0.1	¿hay **una** cosa o **varias**? (sin contar)	`un_objeto` / `varios_objetos`
0.2	¿esos objetos **comparten** algo?	`grupo_por_atributo` / `sin_atributo_comun` / `una_sola_entidad`

La palabra GRUPO es solo de 0.2 y siempre quiere decir: varios objetos que comparten un atributo con el mismo valor. Así ya no se contradicen:

```
casa,carro                                  -> varios_objetos
casa(color=blanco); carro(color=negro)      -> varios objetos, pero sin_atributo_comun
casa_A(color=blanco); casa_B(color=blanco)  -> grupo_por_atributo, comparten color
```

En una línea

Doble clic en `E:\Euclides\PROBAR.bat`. Se abre una ventana, corre tus casos y te deja escribir los tuyos. Para salir, escribe `salir`.

Qué sabe hacer hoy (y qué no)

0.1	Decidir si hay un objeto o varios objetos (sin contarlos)
0.2	**Reconocer** si varios objetos forman grupo y decir **qué atributos comparten**
0.2	**Formar** el grupo de los que cumplen un criterio (`color=rojo`)
0.3	**Pertenencia**: si un objeto concreto pertenece a un grupo (`pertenece` / `no_pertenece` / `no se sabe`)
1.0	**Subitizing**: reconocer de un vistazo la cantidad de un grupo pequeño, sin contar
2.1	**Contar**: emparejar una palabra con cada objeto, en orden (`emparejado`)
2.1	**Revisar una cuenta**: decir si está bien hecha y **cuál** de los tres errores tiene
2.2	**La lista**: qué va antes y después de cualquier palabra, sin empezar por «uno»
2.2	**Al llegar al final dice** `no_lo_se`: no se inventa palabras nuevas

0.1	Decidir si hay un objeto o varios objetos (sin contarlos)
2.3	**Cuántos hay**: cuenta la colección y da la cantidad (la última palabra del recuento)
2.4	**Dame N**: produce un conjunto con esa cantidad (`dame: tres`) y **lo verifica contándolo**
2.4	**¿Tiene N?**: `tiene: a,b,c / tres` → sí o no (y dice cuál tiene)
Siempre	Contesta con **las dos capas**: cuántas cosas hay y si comparten atributos
Siempre	Si no entiende lo que le escribes, **lo dice**; nunca revienta
Siempre	Si no tiene bastante (más objetos que palabras, o un dato que no sabe), **lo dice**; no se lo inventa
No sabe todavía	**Decir cuántos hay** («hay tres»). El sistema empareja, pero aún no sabe que la última palabra es el total (eso es la 2.4). Tampoco compara cantidades ni usa cifras

Las tres formas de preguntar

1. Un objeto o varios (unidad 0.1)

Cosas separadas por **comas**. Si repites un nombre, es **la misma** cosa señalada otra vez.

```
silla                -> un_objeto
silla,mesa           -> varios_objetos
silla,silla          -> un_objeto   (la misma silla, dos veces)
```

2. ¿Forman grupo? (unidad 0.2)

Objetos con atributos. Valen **punto y coma** o **comas de fuera** de los paréntesis. Los atributos van entre paréntesis como `nombre=valor`, separados por comas.

```
caja_A(color=rojo,forma=cubo); caja_B(color=rojo,forma=esfera)   -> grupo por color
caja_A(color=rojo,forma=cubo), caja_B(color=azul,forma=cubo)     -> grupo por forma
casa(color=blanco); carro(color=negro)                           -> sin_atributo_comun
```

3. Dame el grupo de... (unidad 0.2)

El criterio, dos puntos, y la escena.

```
color=rojo: a(color=rojo,forma=cubo); b(color=azul,forma=cubo); c(color=rojo,forma=esfera)
   -> el grupo son a y c
```

4. ¿Pertenece? (unidad 0.3)

Empieza por `pertenece`, luego el criterio del grupo, dos puntos, y el objeto o los objetos.

```
pertenece color=rojo: taza(color=rojo)
   -> taza -> PERTENECE

pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)
   -> taza  -> PERTENECE
```

```
   -> plato -> NO_PERTENECE
   -> vaso  -> NO_DECIDIBLE  (no se sabe su color)
```

5. ¿Cuántos hay, de un vistazo? (unidad 1.0)

Empieza por `subitizar`, dos puntos, y la figura: **tres filas separadas por** `/`, con `x` de elemento y `.` de hueco.

```
subitizar: x.x/.../x.x
   -> es el patron 'esquinas_cuatro' -> CUATRO

subitizar: x../xx./x.x
   -> no_reconocido  (no lo reconoce de un vistazo, y NO lo cuenta)
```

6. Contar (unidad 2.1)

Empieza por `contar`, dos puntos, y la colección (separada por comas).

```
contar: silla_1, silla_2, silla_3
   -> silla_1 -> uno ; silla_2 -> dos ; silla_3 -> tres   (emparejado)

contar: silla_1, silla_1, silla_1
   -> un solo objeto (es el mismo nombre tres veces) -> una sola palabra: uno

contar: casa(color=blanco), casa, perro, casa(forma=cuadrado), carro
   -> casa [color=blanco, forma=cuadrado]  -> uno     (las tres menciones son LA MISMA casa: los
   -> perro                                -> dos      datos se suman, y no dan tres palabras)
   -> carro                                -> tres
```

7. Revisar una cuenta (unidad 2.1)

Empieza por `revisar`, la colección, una **barra** `/`, y las palabras que le pusieron.

```
revisar: a,b,c / uno,dos,tres      -> correcta
revisar: a,b,c / uno,uno,tres      -> palabra_repetida
revisar: a,b,c / uno,tres,cuatro   -> orden_saltado
revisar: a,b,c / uno,dos           -> faltan_objetos_sin_contar
revisar: a,b / uno,once            -> palabra_desconocida
revisar: a,b / uno,dos,tres        -> palabras_de_mas
```

8. La lista: qué va antes y después (unidad 2.2)

```
siguiente: tres     -> cuatro
anterior: tres      -> dos
siguiente: diez     -> no_lo_se    (la lista se acaba; no se inventa «once»)
siguiente: once     -> palabra_desconocida
recitar: ocho       -> ocho, nueve, diez
recitar: uno        -> la lista entera, uno a uno
```

9. Revisar una recitación ajena (unidad 2.2)

```
revisar_secuencia: uno,dos,tres     -> correcta
revisar_secuencia: uno,dos,cuatro   -> orden_saltado
revisar_secuencia: uno,tres,dos     -> orden_saltado
revisar_secuencia: uno,dos,once     -> palabra_desconocida
```

10. Cuántos hay (unidad 2.3)

```
cuantos: cosa                        -> uno
cuantos: silla_1, silla_2, silla_3   -> tres
cuantos: a,b,c,d,e                   -> cinco
cuantos: a,a,a,a,a                   -> uno   (el mismo objeto cinco veces)
cuantos: (once objetos)              -> no_alcanza_la_secuencia
```

11. Dame N (unidad 2.4)

```
dame: tres     ->  cosa_uno, cosa_dos, cosa_tres  (y lo verifica contándolo)
dame: uno      ->  cosa_uno
dame: diez     ->  diez objetos
dame: once     ->  palabra_desconocida
```

12. ¿Tiene N? (unidad 2.4)

```
tiene: a,b,c / tres     -> si
tiene: a,b,c / cinco    -> no   (tiene tres)
tiene: (once objetos) / cinco -> no_alcanza_la_secuencia
```

Lo más importante: el nombre es el identificador

El sistema **no ve** los objetos, y **no adivina** por el parecido de los nombres. El nombre que escribes **es** la identidad de la cosa.

- `silla` y `silla` → **la misma** silla.
- `silla_A` y `silla_B` → **dos** sillas distintas, aunque en el mundo las dos se llamen igual.

Y si escribes la misma cosa con dos descripciones que no pueden ser ciertas a la vez:

```
casa(color=azul); casa(color=rojo)
   -> son DOS objetos distintos: el color los distingue (casa#1 y casa#2)
```

Pero si los datos **no** se contradicen, son la misma cosa y sus datos se **suman**:

```
casa(color=blanco); casa(forma=cuadrada)
   -> un solo objeto: casa blanca y cuadrada
```

Si escribes algo que no entiende

No revienta: te lo dice y te recuerda las formas correctas.

```
pregunta> caja(colorrojo)

NO ENTIENDO ESTO: no entiendo el atributo 'colorrojo'
   se escribe asi:  nombre=valor

FORMAS QUE SI ENTIENDO:
  0.1  un objeto o varios  silla,mesa
  0.2  forman grupo?       a(color=rojo,forma=cubo); b(color=azul,forma=cubo)
  0.2  dame el grupo de... color=rojo: a(color=rojo); b(color=azul)

El sistema no adivina: si no entiende, lo dice.
```

Las tres formas de lanzarlo

1. **Doble clic en** `PROBAR.bat` — lo normal. Corre tus casos y abre la ventana.
2. `py -3 E:\Euclides\probar.py "silla,mesa"` — una pregunta suelta.
3. `py -3 E:\Euclides\probar.py --casos` — todos tus casos de golpe.

Órdenes dentro de la ventana: `casos`, `ayuda`, `salir`.

Dónde escribes TUS casos

Archivo: `E:\Euclides\mis_casos.txt` (Bloc de notas). Cada línea:

```
pregunta | lo que espero | comentario
```

Lo que puedes poner en "lo que espero":

Escribes	Significa
`objeto` o `uno`	una sola entidad
`varios`	dos o más entidades distintas
`grupo:color`	forman grupo y comparten el color
`grupo:color,forma`	comparten las dos cosas
`sin_atributo_comun`	son varios, pero no comparten nada
`una_sola_entidad`	solo hay una cosa (o es la misma descrita dos veces): la pregunta de grupo no aplica
`tiene:a,c`	al formar un grupo, el resultado son esos
`no_hay_grupo`	ninguno cumple el criterio
`no_decidible`	el sistema debe decir que no sabe
`no_entendido`	el sistema debe decir que no entiende la escritura
`taza:pertenece`	(0.3) la taza pertenece al grupo que se pregunta
`plato:no_pertenece`	(0.3) el plato no pertenece
`vaso:no_decidible`	(0.3) del vaso no se sabe: no es lo mismo que no pertenecer
`reconocido:cuatro`	(1.0) la figura es el patrón de cuatro
`no_reconocido`	(1.0) el sistema no reconoce la figura de un vistazo, y no la cuenta
`emparejado`	(2.1) contó: una palabra por objeto, sin repetir ni saltar
`no_alcanza_la_secuencia`	(2.1) hay más objetos que palabras conoce: no se las inventa
`correcta`	(2.1) la cuenta que le dieron está bien hecha
`palabra_repetida`	(2.1) la misma palabra para dos objetos
`orden_saltado`	(2.1) se saltaron una palabra de la secuencia
`faltan_objetos_sin_contar`	(2.1) algún objeto se quedó sin contar
`palabra_desconocida`	(2.1) hay una palabra que no está en la secuencia
`palabras_de_mas`	(2.1) hay más palabras que objetos

Escribes	Significa
`cuatro` / `ocho` / …	(2.2) la palabra que va después (o antes)
`no_lo_se`	(2.2) la lista se acaba ahí: el sistema no se inventa la siguiente
`recitado:ocho,nueve,diez`	(2.2) la recitación desde esa palabra
`uno` / `tres` / `cinco` …	(2.3) la cantidad que hay. **OJO**: aquí `uno` es la cantidad, no «un objeto»
`no_alcanza_la_secuencia`	(2.3) más objetos que palabras conoce: no se inventa la cantidad
`cumplido`	(2.4) `dame: N` ha producido el conjunto y lo ha verificado
`si` / `no`	(2.4) `tiene: <colección> / N` — si el conjunto tiene esa cantidad o no
`?`	no comprobar nada, solo ver qué responde

Guarda el archivo y vuelve a lanzar el probador: **se recarga solo**. Los # son comentarios.

Por qué el sistema explica lo que dice

Cada respuesta sale con su **traza**: qué reglas se han probado, en qué orden, cuál ha decidido y qué conocimiento escrito la respalda. Ejemplo real:

```
PREGUNTA (0.2): forman GRUPO estos objetos?
ESCENA:
    casa  [color=blanco]
    carro  [color=negro]
PLURALIDAD (0.1): son VARIOS OBJETOS
  reglas probadas, en orden:
    - reconocer_escena_vacia : escena_vacia(escena) -> falso
    - reconocer_una_sola_entidad : menos_de_dos_entidades(escena) -> falso
    - reconocer_atributo_indeterminado : algun_atributo_indeterminado(escena) -> falso
    - reconocer_grupo_por_atributo : hay_atributo_compartido_por_todos(escena) -> falso
    - reconocer_sin_atributo_comun : sin_atributo_compartido_por_todos(escena) -> cierto
ATRIBUTOS (0.2): sin_atributo_comun
  lo decide la regla 'reconocer_sin_atributo_comun'
  respaldado por el conocimiento 'grupo_por_atributo' (Grupo por atributo) [FIJADO]
```

Qué buscar cuando lo pruebes

1. **Que 0.1 y 0.2 ya no se contradigan**: `casa,carro` contra

`casa(color=blanco); carro(color=negro)`.

2. **La pertenencia y sus tres respuestas**: `pertenece color=rojo: taza(color=rojo); plato(color=azul); vaso(forma=cubo)`.

Comprueba que el vaso dice **«no se sabe»** y no «no pertenece».

3. **El mismo objeto en dos grupos**: `pertenece color=verde:` y `pertenece forma=triangulo:` sobre

`triangulo(color=verde,forma=triangulo)`. Debería pertenecer a los dos.

4. **La fusión y la distinción**: `casa(color=blanco); casa(forma=cuadrada)` debería decir que es la

misma casa **con los datos sumados; y** `carro(color=negro); carro(color=blanco)` **debería dar** dos

objetos (`carro#1`, `carro#2`), no «no se sabe».

5. **Cosas escritas a medias o mal** (`caja(colorrojo)`, `caja(color=)`, `pertenece =rojo: ...`): todas

deberían contestar "no entiendo", ninguna debería romperse.

6. **La misma cosa con dos descripciones distintas que sí chocan** (`casa(color=azul); casa(color=rojo)`).
7. **Un objeto al que le falta un atributo** que los demás sí tienen: debería callarse **en atributos**

pero seguir diciendo que son varios objetos.

8. **Repetir el mismo nombre** en 0.1 (`taza,taza,taza`).
9. **Pedir un grupo que no existe** (`color=verde:` cuando nadie es verde).
10. **El subitizing y su negativa**: `subitizar: x.x/.../x.x` debería dar **cuatro**; y

`subitizar: x../xx./x.x` (que también tiene cuatro) debería dar **no_reconocido** — porque el

sistema **no cuenta**. Ese "no" es la respuesta correcta, no un fallo.

11. **(2.1) Que cuente una colección**: `contar: a,b,c` debería emparejar uno, dos y tres.
12. **(2.1) La prueba que encontraste tú**: `contar: casa(color=blanco), casa, perro,

casa(forma=cuadrado), carro` debería dar **tres** palabras (casa, perro, carro), y la casa con sus

dos datos sumados. Y el contraste: `contar: casa(color=blanco), casa(color=rojo), carro` sí son

dos casas, porque el color las distingue.

13. **(2.1) Que cace los tres errores**, uno por uno:

`revisar: a,b,c / uno,uno,tres` (repetida — el más común), `revisar: a,b,c / uno,tres,cuatro`

(salto) y `revisar: a,b,c / uno,dos` (un objeto sin contar). Y que diga **cuál** es.

14. **(2.1) Que NO se invente palabras**: `contar:` con once objetos debería decir

`no_alcanza_la_secuencia`, no seguir contando.

15. **(2.1) Que la cuenta correcta se reconozca como correcta**: `revisar: a,b,c / uno,dos,tres`.
16. **(2.2) Que sepa la lista sin empezar por uno**: `siguiente: siete` → ocho, y `anterior: nueve`

→ ocho.

17. **(2.2) Que al llegar al final NO se invente nada**: `siguiente: diez` → `no_lo_se`, y

`siguiente: once` → `palabra_desconocida`. Ese «no lo sé» es la respuesta correcta.

18. **(2.2) Que cace una recitación mal dicha**: `revisar_secuencia: uno,dos,cuatro` → `orden_saltado`

y que diga que falta el tres; `revisar_secuencia: uno,dos,once` → `palabra_desconocida`.

19. **(2.3) Que diga cuántos hay**: `cuantos: silla_1, silla_2, silla_3` → `tres`;

`cuantos: a,b,c,d,e` → `cinco`; `cuantos: cosa` → uno.

20. **(2.3) Que no cuente dos veces el mismo objeto**: `cuantos: a,a,a,a,a` → uno.
21. **(2.3) Que no se invente la cantidad**: `cuantos:` con once objetos → `no_alcanza_la_secuencia`.

22. **(2.4) Que sepa dar**: `dame: uno`, `dame: tres`, `dame: diez` → produce el conjunto y lo verifica; `dame: once` → `palabra_desconocida`.
23. **(2.4) Que confirme y rechace**: `tiene: a,b,c / tres` → `si`; `tiene: a,b,c / cinco` → `no`.
24. Cosas tuyas: nombres con espacios, mayúsculas y minúsculas, números (`caja1`, `caja2`).

Si algo te chirría, dímelo con **la línea exacta que escribiste y lo que esperabas**.

Los archivos, por si quieres mirar

Archivo	Qué es
`E:\Euclides\conocimiento\*.kn`	El saber, en texto plano. Se puede leer y editar
`E:\Euclides\sistema\motor.py`	El programa que lee, decide y explica
`E:\Euclides\pruebas\`	Las pruebas del proyecto (0.1, 0.2, 0.3, 1.0, 2.1 y robustez)
`E:\Euclides\mis_casos.txt`	**Tus casos**
`E:\Euclides\PROBAR.bat`	El lanzador de doble clic
`E:\Euclides\PROGRESO.md`	Qué está hecho, qué falla, qué toca
`E:\Euclides\diario\`	Lo que se hizo cada día y lo que se midió